Deadline-Driven Scheduler

Nolan Curtis - V009

S M Ashraf - V009

ECE 455

2025-04-07

Introduction

The objective of this project is to create a Deadline-Driven-Scheduler(DDS) to manage tasks that have hard execution deadlines. The project presents the design and implementation of DDS built on top of FreeRTOS. The DDS system dynamically schedules real-time tasks using the earliest deadline first (EDF) algorithm. The behaviour of the system is represented in Figure 1 below. The report details all the F-Tasks, queues, DDS functions, and priority levels used as well as the interactions and implementations of each.

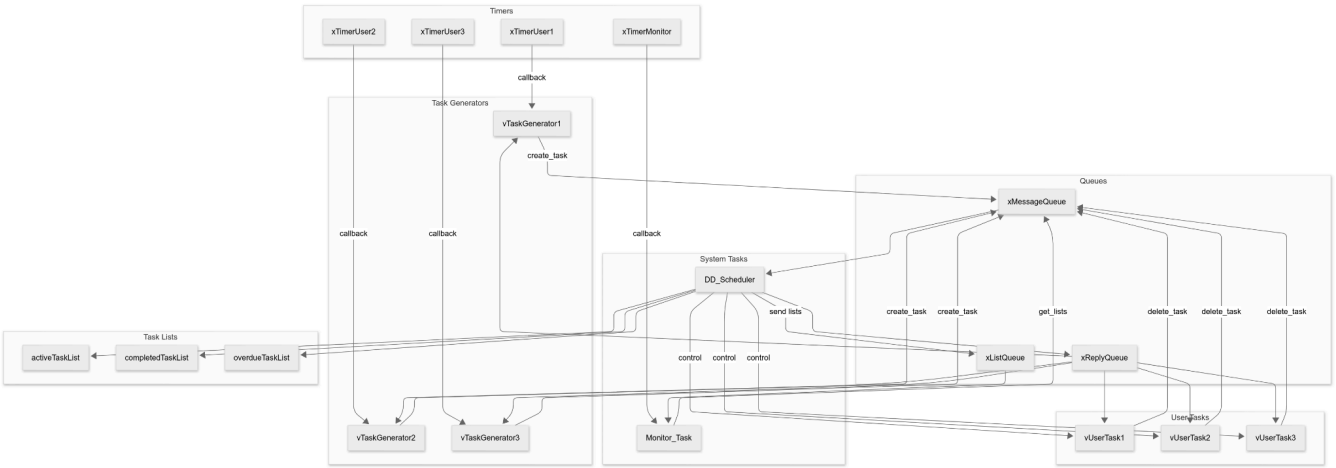

Figure 1: System overview diagram which shows the interactions between F-Tasks, queues, and DDS functions.

Design Solution

Lists

- Active
- Completed
- Overdue

Tasks

- DDS task at maximum priority for handling task priority management
- Monitor task for logging active/completed/overdue tasks
- User tasks - executes for time period
- Generator tasks - makes user tasks, based on period

Queues

- Message queue - send messages to scheduler
- Reply queue - reply from scheduler

Logic

DDS scheduler adds user tasks, which have a deadline, generated by generator tasks, which have a period, into the active list, DD scheduler assigns priority. DD scheduler moves

tasks and detects overdue and completed tasks. Monitor tasks checks all 3 lists and reports back the current state, its at a high priority so it doesnt get superceded by other tasks. DD scheduler should be at highest priority as it should never be superceded, then monitor task, then user tasks or generator tasks.

Differences

The differences we had between our design document and our implementation is that we missed out on specifying certain functions and lists. We needed to add a list queue for monitor task to get the task list, and additional functions were made for creation and deletion of tasks.

F-Tasks

DD scheduler	4
Monitor task	3
Task generator	2
User task	1

DD_Scheduler - Maximum priority (4)

This is the central controlling task that sits at the highest priority level, ensuring it can preempt any other task in the system when necessary. The scheduler's primary responsibilities include:

- **Task List Management**: Maintains three crucial linked lists (activeTaskList, completedTaskList, and overdueTaskList) that track the status of every task in the system. Each list node contains comprehensive task metadata including handles, deadlines, release times, and completion times.
- **Message Processing**: Constantly monitors the xMessageQueue for incoming requests from other components. The scheduler processes five distinct message types (create, delete, active, completed, overdue) and takes appropriate actions for each.
- **Scheduling Algorithm**: Implements the Earliest Deadline First (EDF) scheduling algorithm. When a new task arrives, the scheduler inserts it into the activeTaskList in deadline order, ensuring tasks with earlier deadlines are executed first.
- **Task Control**: Has complete authority to suspend, resume, and adjust the priorities of user tasks. When a task becomes the earliest deadline task, the scheduler raises its priority to PRIORITY_HIGH and resumes it, allowing it to execute.

- **Deadline Monitoring**: Tracks whether tasks complete before or after their deadlines and moves them to the appropriate list (completedTaskList or overdueTaskList) when they finish execution.

Monitor_Task- High Priority (3)

This task runs at slightly lower priority than the scheduler but higher than all other tasks, allowing it to gather system information without disrupting the scheduler's operation. Its functions include:

- **Periodic Status Updates**: Awakens every MONITOR period (500 ms) to provide a snapshot of the system's current state. This periodic nature is controlled by the xTimerMonitor timer.
- **Task List Inspection**: Requests and displays statistics from all three task lists maintained by the scheduler. It doesn't modify the lists but only observes them, acting as a non-intrusive observer.
- **Performance Tracking**: By displaying task counts in each list, it provides real-time feedback on system performance. A growing overdue list, for instance, would indicate that the system is becoming overloaded.
- **Self-Regulation**: After gathering and displaying information, it suspends itself until the next timer expiration, minimizing its impact on system resources.

Task Generators (vTaskGenerator1, vTaskGenerator2, vTaskGenerator3) - Medium Priority (2)

These three tasks are responsible for creating new instances of their respective user tasks:

- **Periodic Activation**: Each generator is activated by its corresponding timer (xTimerUser1, xTimerUser2, xTimerUser3). When activated, it creates a new instance of its user task with appropriate timing parameters.
- **Task ID Management**: Each generator maintains and increments a unique task ID counter (taskID1, taskID2, taskID3) to distinguish between different instances of the same user task.
- **Deadline Calculation**: Computes absolute deadlines for tasks based on the period multiplied by the number of instances created. This establishes the time by which each task instance must complete.
- **Scheduler Interaction**: Communicates with the DD_Scheduler through the message queue system, sending create messages and waiting for acknowledgments before suspending itself.

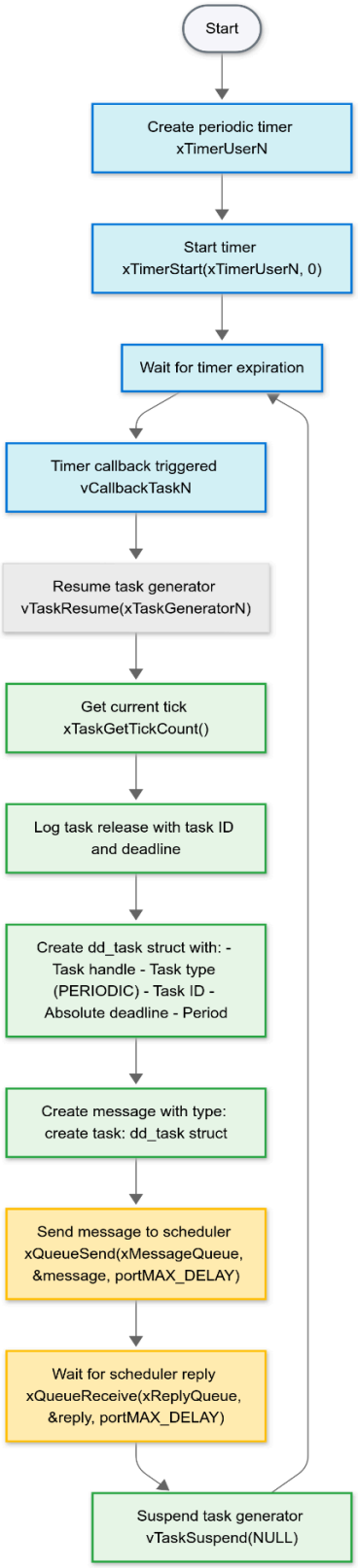

Figure 2: Flow chart of how the Deadline-Task Generator works.

Active User Tasks

When a user task is currently being executed, it temporarily runs at PRIORITY_HIGH (4):

- **Dynamic Priority Adjustment**: The scheduler raises the priority of the task with the earliest deadline to PRIORITY_HIGH, allowing it to preempt other, lower-priority tasks.
- **Execution Control**: Only one user task runs at PRIORITY_HIGH at any given time. When a higher-priority task arrives (earlier deadline), the scheduler suspends the current task, returns its priority to PRIORITY_MIN, and promotes the new task.
- **Execution Timeframe**: The task remains at PRIORITY_HIGH until it either completes its execution or is preempted by a task with an earlier deadline.

User Tasks (vUserTask1, vUserTask2, vUserTask3) - Low Priority (1)

These tasks represent the actual workloads being scheduled and have these detailed characteristics:

- **Execution Time Simulation**: Each task simulates a specific workload by running for a predetermined time (T1_EXEC, T2_EXEC, T3_EXEC). This is accomplished through a busy-wait loop that consumes processor cycles for the specified duration.
- **Deadline-Aware Operation**: While the tasks themselves don't directly compute deadlines, they're designed to work within the deadline-driven framework. Each task reports its completion time, which the scheduler compares against the task's deadline.
- **Task-Specific Parameters**: Each task has unique execution time and period parameters based on the selected benchmark (Bench_Choice):
    - **Benchmark 1**: Balanced loads with tasks of 95ms, 150ms, and 250ms execution times and periods of 500ms, 500ms, and 750ms respectively.
    - **Benchmark 2**: More challenging timing with shorter periods for Task 1 (250ms) while maintaining the same execution times.
    - **Benchmark 3**: Equal periods (500ms) with balanced execution times (100ms, 200ms, 200ms).
- **Performance Verification**: Each task contains an expected completion time array that allows the system to verify whether the scheduler is performing correctly. The task compares its actual completion time against these expected values and reports the comparison.
- **Completion Notification**: When a task finishes its simulated workload, it notifies the DD_Scheduler by calling delete_dd_task with its task ID, allowing the scheduler to update its lists and schedule the next task.

This multi-level priority structure, combined with the deadline-driven scheduling algorithm, creates a sophisticated real-time scheduling system.

Functions

DD_Scheduler

The DD_Scheduler function serves as the core component of the entire DDS system. This high-priority task manages the scheduling of all user tasks according to their deadlines.

The function begins by initializing three critical linked lists: activeTaskList, completedTaskList, and overdueTaskList. Each list is dynamically allocated using malloc, and its head node is initialized with an empty task structure created by calling the helper function allocateEmptyTask(). This empty structure has NULL or zero values for all fields, serving as a sentinel node.

After initialization, the scheduler enters an infinite loop where it continually monitors the message queue (xMessageQueue) for incoming messages using xQueueReceive() with portMAX_DELAY, meaning it will block indefinitely until a message arrives.

When a message is received, the scheduler first captures the current system tick count using xTaskGetTickCount(). If there's currently a task running (indicated by a non-NULL task handle in the active list), the scheduler suspends it with vTaskSuspend() and resets its priority to PRIORITY_MIN. This ensures the scheduler has complete control over which task executes and when.

The scheduler then processes the message based on its type:

For create messages, the scheduler:

1. Allocates a new node for the task
2. Checks if the task is schedulable by comparing its absolute deadline with the current time plus period
3. If not schedulable, adds it to the overdue list
4. If schedulable, finds the correct position in the active list based on earliest deadline first
5. Performs detailed pointer manipulation to insert the node in the proper position, handling edge cases like empty lists, insertions at the beginning, middle, or end
6. Sends a reply back through xReplyQueue.

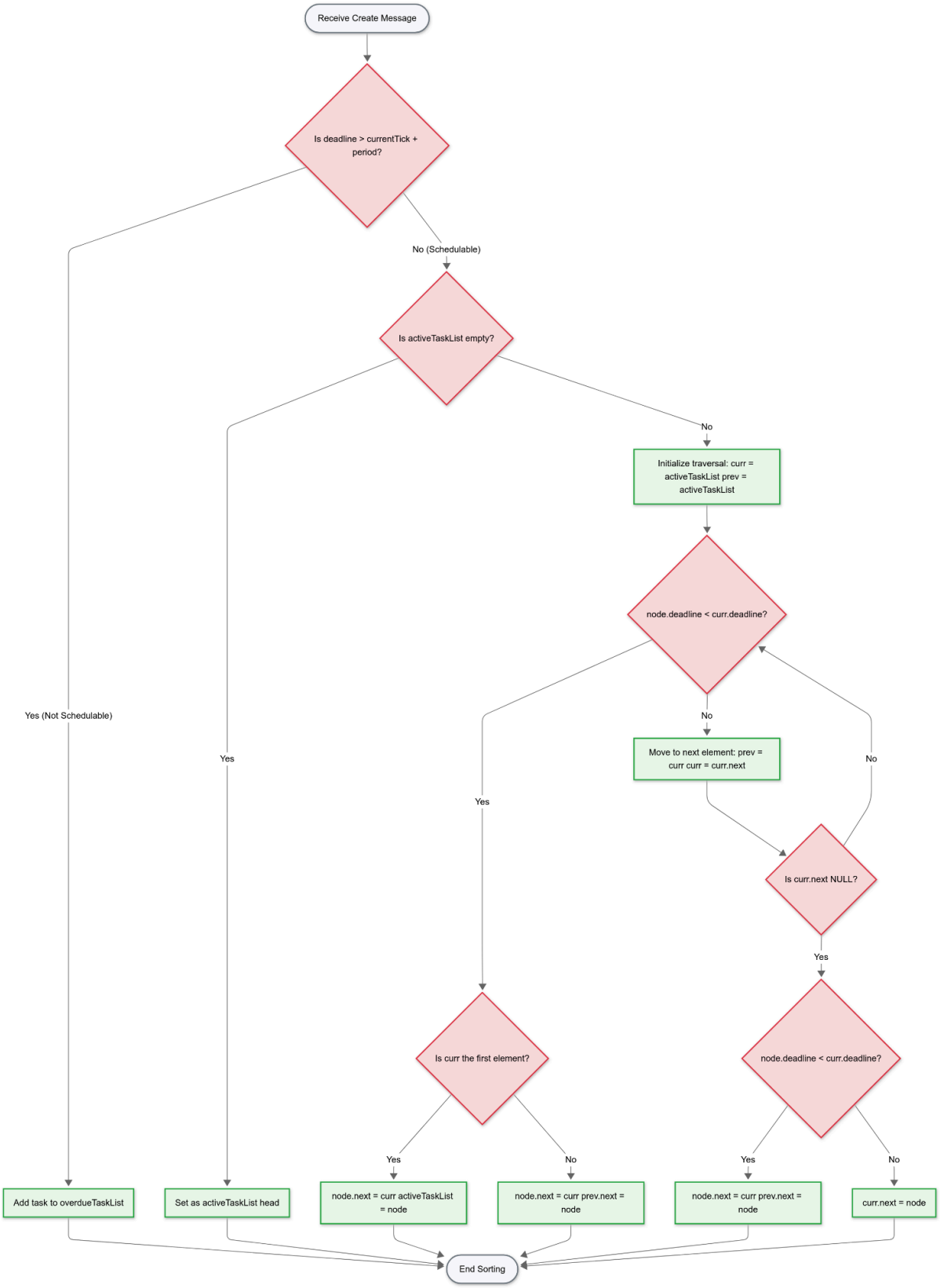

Figure 3: Flow chart of the DD-Task sorting algorithm

For delete messages, the scheduler:

1. Sets the completion time of the current active task to the current tick
2. Determines if the task met its deadline by comparing completion time with absolute deadline
3. If completed on time, moves it to the completed list
4. If overdue, moves it to the overdue list
5. Updates the active list by promoting the next task (if any)
6. Sends a reply back through xReplyQueue

For list query messages (active, completed, overdue), the scheduler:

1. Sends the appropriate list pointer through xListQueue
2. Resumes the monitor task with vTaskResume()

After processing the message, if there's a task in the active list (deadline > 0), the scheduler:

1. Updates the release time if it's not yet set
2. Raises the task's priority to PRIORITY_HIGH using vTaskPrioritySet()
3. Resumes the task with vTaskResume()

The scheduler then returns to the beginning of the loop to wait for the next message.

create_dd_task

The create_dd_task function is the interface used by task generators to create and register tasks with the scheduler. It takes five parameters: a task handle, task type (PERIODIC or APERIODIC), task ID, absolute deadline, and period.

The function begins by creating a dd_task structure and filling it with the provided parameters. It initializes the release time and completion time to 0, as these will be set by the scheduler when the task starts and finishes execution.

It then wraps this task structure in a dd_message with the type set to create. The message is sent to the scheduler via xQueueSend() with portMAX_DELAY, ensuring the function blocks until the message is successfully queued.

After sending the message, the function waits for a reply from the scheduler by calling xQueueReceive() on xReplyQueue. This synchronization ensures that the task creation is properly acknowledged before proceeding.

Finally, the function suspends itself using vTaskSuspend(NULL), allowing the scheduler to take control. This is crucial because task generators run at PRIORITY_HIGH, and without this self-suspension, they would continue to run and potentially interfere with task scheduling.

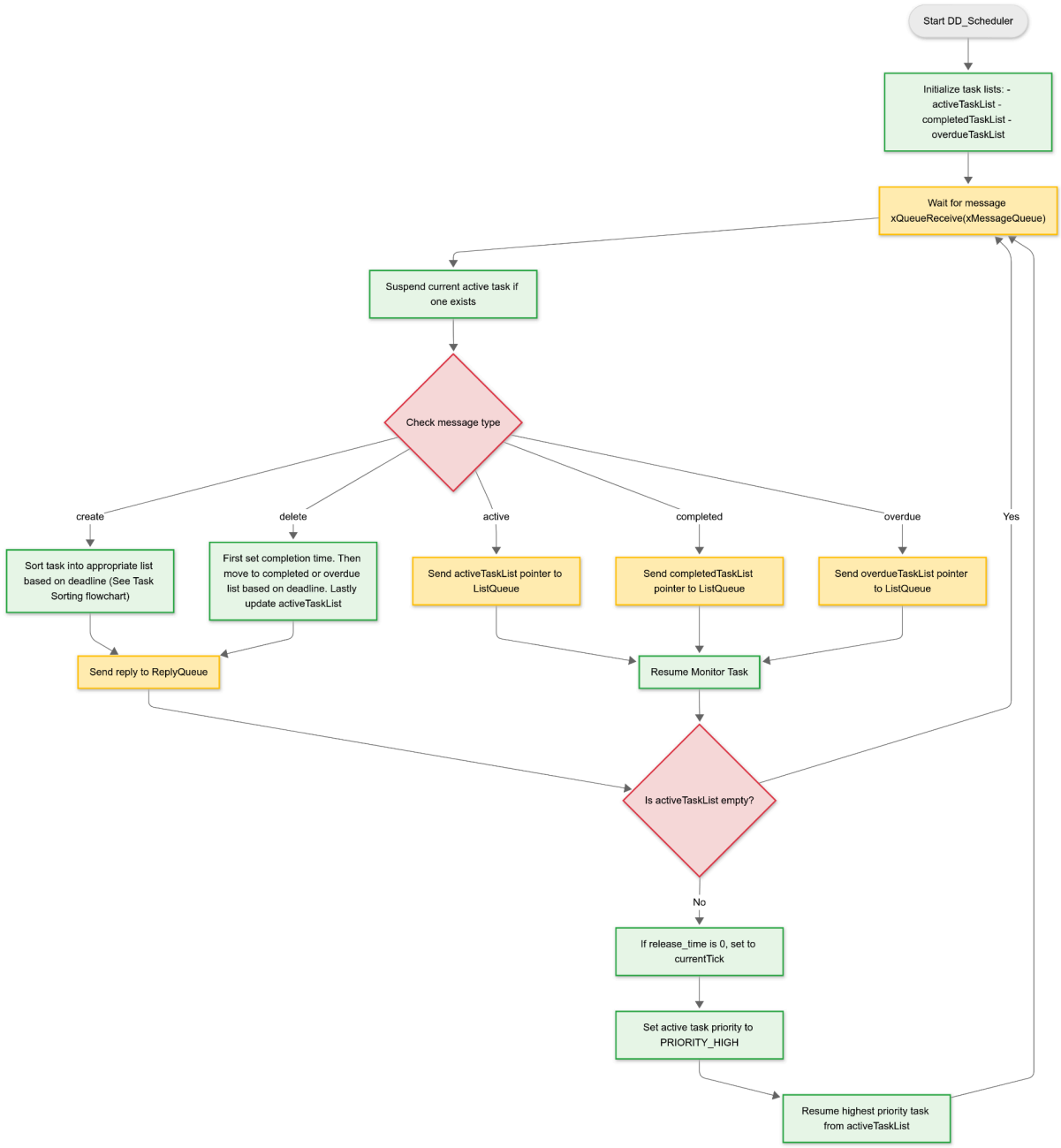

Figure 4: Flow chart of how the DD Scheduler works.

delete_dd_task

The delete_dd_task function is called by user tasks when they complete their execution. It takes a single parameter: the task ID to be deleted.

The function creates a minimal dd_task structure with only the task ID field set and wraps it in a dd_message with the type set to delete. This message is then sent to the scheduler via xQueueSend().

Similar to create_dd_task, this function waits for an acknowledgment from the scheduler by calling xQueueReceive() on xReplyQueue. This ensures that the task deletion is properly processed before allowing the task to continue.

Unlike create_dd_task, this function does not suspend itself at the end because the task has completed its work cycle and is ready to be rescheduled if needed.

get_active_dd_task_list

This function is used by the Monitor_Task to retrieve the active task list from the scheduler. It begins by creating a **dd_message** with the type set to **active** and sends it to the scheduler via **xQueueSend()**.

The function then waits to receive a pointer to the active task list from **xListQueue**. Once received, it traverses the list to count the number of tasks:

1. It starts with the head node of the list
2. It counts the number of non-empty tasks by traversing the linked list
3. It checks if the current node has a valid task ID (not 0) to include it in the count
4. It prints the count to the console

The function returns the pointer to the active list, allowing the monitor task to access the detailed information if needed.

get_completed_dd_task_list

This function follows the same pattern as get_active_dd_task_list but with the message type set to completed. It requests the completed task list from the scheduler, counts the tasks, and displays the count.

A notable difference in this implementation is the use of a flag variable to handle the sentinel node at the head of the list. The first node is skipped in the count since it's an empty placeholder, and only subsequent nodes represent actual completed tasks.

get_overdue_dd_task_list

Similar to the other list retrieval functions, this one retrieves and counts the tasks in the overdue list. It follows the same pattern of sending a message, receiving the list pointer, traversing and counting tasks, and printing the count.

The function also uses a flag to skip the sentinel node in the count, ensuring only actual overdue tasks are counted. After printing, it adds an extra newline to separate the monitor output from subsequent system messages.

allocateEmptyTask Function (Helper)

This utility function creates and returns an empty dd_task structure with all fields initialized to NULL or 0. It's used to create sentinel nodes for the linked lists and to initialize task structures before they're populated with actual values.

The empty task has a NULL task handle, PERIODIC type (as default), and all numeric fields set to 0. This consistent initialization prevents undefined behavior when the scheduler processes tasks.

Each of these functions plays a vital role in the DDS system, working together to implement a deadline-driven scheduler on top of FreeRTOS's native priority-based scheduler. The design demonstrates how more sophisticated scheduling policies can be implemented on embedded platforms with limited resources, providing predictable real-time behavior for tasks with varying execution times and deadlines.

Discussion of DDS scheduler performance

Scheduling Performance

Timelines

Both test benches show generally good adherence to expected completion times, but as time goes on, Test Bench 2 starts to have a few task enter the overdue queue. This is probably due to the tighter deadlines. The measured completion times closely match the expected completion times with minor deviations of 2-5ms, indicating low scheduling overhead. This suggests the scheduler implementation is efficient with minimal context switching costs.

Task Completion

In Test Bench 1, all tasks consistently meet their deadlines. No overdue tasks reported in any Monitor Task outputs. Completed task count increasing steadily (3, 5, 8, 11) and active tasks count remaining low (0 or 1). This is perhaps due to the periods of the tasks being sufficiently long for them to completion time. Below shows the table of DDS events generated for Task Bench #1.

```
Event #      Event       Measured Time (ms)      Expected Time (ms)
=============================================================================
=
    1      Task 1 released        2           0
    2      Task 2 released        3           0
    3      Task 3 released        4           0
    4      Task 1 complete        99              95
    5      Task 2 complete        249             245
    6      Task 3 complete        499             495

           Monitor Task:          500
           Active:  0
           Completed:   3
           Overdue:     0

    7      Task 1 released        501             500
    8      Task 2 released        502             500
    9      Task 1 complete        598             595
    10     Task 2 complete        748             745
    11     Task 3 released        750             750

           Monitor Task:          1000
           Active:  1
           Completed:   5
           Overdue:     0
```

Figure 5: DDS events generated for Task Bench #1

In Test Bench 2, we see a different pattern due to the higher workload as Task 1 appears more frequently in the execution sequence. The system maintains more active tasks at certain points, and after about 3000ms, we started to notice tasks being added to the overdue queue. This is perhaps due to Task 1 having a shorter period. This would add pressure on the scheduler to make sure the tasks are done on time, but eventually it starts missing deadlines, and those tasks get added to the overdue queue.

```
           Monitor Task:          1000
           Active:  1
           Completed:   7
           Overdue:     0
```

```
           Monitor Task:          1500
           Active:  1
           Completed:   10
           Overdue:     0
```

Figure 6a (above) and 6b (below): Monitor Task outputs for Task Bench #2

Schedulability and Utilization

The total CPU utilization can be calculated as:

- Test Bench 1: (95/500 + 150/500 + 250/750) = 0.19 + 0.3 + 0.33 = 0.82 (82%)
- Test Bench 2: (95/250 + 150/500 + 250/750) = 0.38 + 0.3 + 0.33 = 1.01 (101%)

Theoretically, with EDF scheduling, a task set is schedulable if the total utilization is less than or equal to 1. Test Bench 2 is right at the theoretical limit, yet still manages to complete most tasks without any overdue reports, but eventually it starts missing tasks. Regardless, it is still impressive.

Scheduler Algorithm Effectiveness

The DD_Scheduler function in the code manages tasks based on their absolute deadlines. When examining the execution sequence in the test results:

1. The scheduler successfully prioritizes tasks with earlier deadlines
2. It maintains proper task queues (active, completed, overdue)
3. It handles task preemption correctly by suspending the current task when a higher priority task arrives

In Test Bench 2, we can observe the scheduler's ability to handle higher workloads, where Task 1 executes more frequently. The task completion sequence shows that the scheduler correctly prioritizes tasks according to their deadlines.

Implementation Strengths

1. **Robust List Management**: The scheduler maintains separate lists for active, completed, and overdue tasks, allowing for effective task tracking and monitoring.
2. **Efficient Priority Mechanism**: The implementation leverages FreeRTOS's priority system by setting active tasks to PRIORITY_HIGH (2) while keeping inactive tasks at PRIORITY_MIN (1).
3. **Effective Message Passing**: The use of queues for communication between tasks and the scheduler provides a clean interface and reduces coupling.
4. **Monitoring Capability**: The Monitor_Task provides regular snapshots of system state, which is valuable for debugging and performance analysis.

Limitations and Potential Improvements

1. **Memory Management**: The implementation allocates memory for new task list nodes but doesn't necessarily free this memory, which could lead to memory leaks over time.
2. **Error Handling**: There's limited error handling for cases like queue overflow or memory allocation failures.

3. **Schedulability Analysis**: The scheduler doesn't perform admission control based on formal schedulability tests, which could be beneficial for rejecting tasks that would cause the system to miss deadlines.
4. **Dynamic Priority Adjustments**: While the system assigns tasks to higher priority when active, it could potentially benefit from more dynamic priority adjustments based on urgency as deadlines approach.

Overall, the DDS implementation demonstrates effective deadline-based scheduling with good performance characteristics under both normal and high utilization scenarios.

Summary

The DDS project successfully demonstrates a real-time scheduling system using FreeRTOS and Earliest Deadline First (EDF) principles. Tasks are prioritized and managed based on their deadlines, and the scheduler controls execution by suspending and resuming tasks to ensure the most critical ones are prioritized. A dedicated monitor task provides runtime visibility into system performance, enabling detection of missed deadlines through overdue task accumulation. The system uses message queues to facilitate clean and effective task creation, deletion, and monitoring, supporting reliable task synchronization and state management. Future enhancements could adapt this scheduler for real-world applications where meeting deadlines is critical for safety and performance.

Appendix

A.1: Source Code Main.c

Main.c

```
/* Standard includes. */
#include <stdint.h>
#include <stdio.h>
#include "stm32f4_discovery.h"
/* Kernel includes. */
#include "stm32f4xx.h"
#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

#define mainREGION_1_SIZE                   8201
#define mainREGION_2_SIZE                   23905
#define mainREGION_3_SIZE                   16807
#define mainCREATE_SIMPLE_BLINKY_DEMO_ONLY    1

static void prvInitialiseHeap( void );

extern void main_DDS( void );


int main( void )
   {
   prvInitialiseHeap();

   #if ( mainCREATE_SIMPLE_BLINKY_DEMO_ONLY == 1 )
      {
         main_DDS();
      }
   #else
      {
         //main_full();
      }
   #endif /* if ( mainCREATE_SIMPLE_BLINKY_DEMO_ONLY == 1 ) */

   return 0;
}



  static void prvInitialiseHeap( void )
```

```
   {
   /* The Windows demo could create one large heap region, in which case it would
    * be appropriate to use heap_4.  However, purely for demonstration purposes,
    * heap_5 is used instead, so start by defining some heap regions.  No
    * initialisation is required when any other heap implementation is used.  See
    * http://www.freertos.org/a00111.html for more information.
    *
    * The xHeapRegions structure requires the regions to be defined in start address
    * order, so this just creates one big array, then populates the structure with
    * offsets into the array - with gaps in between and messy alignment just for test
    * purposes. */
      static uint8_t ucHeap[ configTOTAL_HEAP_SIZE ];
      volatile uint32_t ulAdditionalOffset = 19; /* Just to prevent 'condition is always true'
warnings in configASSERT(). */
      const HeapRegion_t xHeapRegions[] =
      {
         /* Start address with dummy offsets                                        Size */
         { ucHeap + 1,                                mainREGION_1_SIZE },
         { ucHeap + 15 + mainREGION_1_SIZE,           mainREGION_2_SIZE },
         { ucHeap + 19 + mainREGION_1_SIZE + mainREGION_2_SIZE,
mainREGION_3_SIZE },
         { NULL,                                0                 }
      };

      /* Sanity check that the sizes and offsets defined actually fit into the
       * array. */
      configASSERT( ( ulAdditionalOffset + mainREGION_1_SIZE + mainREGION_2_SIZE
+ mainREGION_3_SIZE ) < configTOTAL_HEAP_SIZE );

      /* Prevent compiler warnings when configASSERT() is not defined. */
      ( void ) ulAdditionalOffset;

      vPortDefineHeapRegions( xHeapRegions );
   }




/*-----------------------------------------------------------*/

void vApplicationMallocFailedHook( void )
{
        /* The malloc failed hook is enabled by setting
        configUSE_MALLOC_FAILED_HOOK to 1 in FreeRTOSConfig.h.

        Called if a call to pvPortMalloc() fails because there is insufficient
        free memory available in the FreeRTOS heap.  pvPortMalloc() is called
        internally by FreeRTOS API functions that create tasks, queues, software
```

```
	timers, and semaphores.  The size of the FreeRTOS heap is set by the
	configTOTAL_HEAP_SIZE configuration constant in FreeRTOSConfig.h. */
	for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationStackOverflowHook( xTaskHandle pxTask, signed char *pcTaskName )
{
	( void ) pcTaskName;
	( void ) pxTask;

	/* Run time stack overflow checking is performed if
	configconfigCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2.  This hook
	function is called if a stack overflow is detected.  pxCurrentTCB can be
	inspected in the debugger if the task name passed into this function is
	corrupt. */
	for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationIdleHook( void )
{
volatile size_t xFreeStackSpace;

	/* The idle task hook is enabled by setting configUSE_IDLE_HOOK to 1 in
	FreeRTOSConfig.h.

	This function is called on each cycle of the idle task.  In this case it
	does nothing useful, other than report the amount of FreeRTOS heap that
	remains unallocated. */
	xFreeStackSpace = xPortGetFreeHeapSize();

	if( xFreeStackSpace > 100 )
	{
		/* By now, the kernel has allocated everything it is going to, so
		if there is a lot of heap remaining unallocated then
		the value of configTOTAL_HEAP_SIZE in FreeRTOSConfig.h can be
		reduced accordingly. */
	}
}
/*-----------------------------------------------------------*/

static void prvSetupHardware( void )
{
	/* Ensure all priority bits are assigned as preemption priority bits.
	http://www.freertos.org/RTOS-Cortex-M3-M4.html */
	NVIC_SetPriorityGrouping( 0 );
```

```
        /* TODO: Setup the clocks, etc. here, if they were not configured before
        main() was called. */
}
```

A.2: Source Code Main_DDS.c

Main_DDS.c

```
/* Standard includes. */
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

#include "stm32f4_discovery.h"
/* Kernel includes. */
#include "stm32f4xx.h"
/* Kernel includes. */
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "timers.h"
#include "semphr.h"

/* Priorities at which the tasks are created. */

#define PRIORITY_MAX                4
#define PRIORITY_HIGHMAX        3
#define PRIORITY_HIGH               2
#define PRIORITY_MIN                1

/* The rate at which data is sent to the queue, specified in milliseconds. */
#define MONITOR                                 pdMS_TO_TICKS(500)


/* This demo allows for users to perform actions with the keyboard. */
#define mainNO_KEY_PRESS_VALUE              ( -1 )
#define mainRESET_TIMER_KEY                 ( 'r' )

/* A software timer that is started from the tick hook. */
static TimerHandle_t xTimer = NULL;

/*-----------------------------------------------------------*/
//Bench_Choice 1,2,3
#define Bench_Choice 2

/*-----------------------------------------------------------*/

/* The task that is created in this file. */

typedef enum task_type task_type;

enum task_type {
    PERIODIC,
```

```
    APERIODIC
};

typedef struct dd_task dd_task;

struct dd_task {
    TaskHandle_t t_handle;
    task_type type;
    uint32_t task_id;
    uint32_t release_time;
    uint32_t absolute_deadline;
    uint32_t completion_time;
    uint32_t period;
};

typedef struct dd_task_list dd_task_list;

struct dd_task_list {
    dd_task task;
    struct dd_task_list* next_task;
};

/*-----------------------------------------------------------*/
/*message struct*/

typedef enum message_type message_type;

enum message_type {
    create,
    delete,
    active,
    completed,
    overdue,
};

typedef struct dd_message dd_message;

struct dd_message {
    message_type type;
    dd_task task;
};


/*-----------------------------------------------------------*/
/*Queue's Handles*/

xQueueHandle xMessageQueue;
xQueueHandle xReplyQueue;
```

```
xQueueHandle xListQueue;

/*-----------------------------------------------------------*/
/*Task Handles*/

TaskHandle_t xDDS;
TaskHandle_t xMonitor;

void DD_Scheduler(void* pvParameters);
void Monitor_Task(void* pvParameters);

/*-----------------------------------------------------------*/
/*Timer Handles*/

void create_dd_task(TaskHandle_t t_handle,
    task_type type,
    uint32_t task_id,
    uint32_t absolute_deadline,
    uint32_t period
);

void delete_dd_task(uint32_t task_id);

dd_task_list** get_active_dd_task_list(void);
dd_task_list** get_completed_dd_task_list(void);
dd_task_list** get_overdue_dd_task_list(void);

void DDSInit(void);
void TestInit(void);

void vCallbackTask1(TimerHandle_t xTimer);
void vCallbackTask2(TimerHandle_t xTimer);
void vCallbackTask3(TimerHandle_t xTimer);
void vCallbackMonitor(TimerHandle_t xTimer);

TimerHandle_t xTimerUser1;
TimerHandle_t xTimerUser2;
TimerHandle_t xTimerUser3;
TimerHandle_t xTimerMonitor;

/*-----------------------------------------------------------*/
/*User Tasks*/
void vUserTask1(void* pvParameters);
void vUserTask2(void* pvParameters);
void vUserTask3(void* pvParameters);

void vTaskGenerator1(void* pvParameters);
void vTaskGenerator2(void* pvParameters);
```

```
void vTaskGenerator3(void* pvParameters);

/*-----------------------------------------------------------*/
/*Task Handles*/

TaskHandle_t xUserTask1;
TaskHandle_t xUserTask2;
TaskHandle_t xUserTask3;

TaskHandle_t xTaskGenerator1;
TaskHandle_t xTaskGenerator2;
TaskHandle_t xTaskGenerator3;


#if Bench_Choice == 1
#define T1_EXEC                         pdMS_TO_TICKS(95)
#define T1_PERIOD                   pdMS_TO_TICKS(500)

#define T2_EXEC                         pdMS_TO_TICKS(150)
#define T2_PERIOD                   pdMS_TO_TICKS(500)

#define T3_EXEC                         pdMS_TO_TICKS(250)
#define T3_PERIOD                   pdMS_TO_TICKS(750)

int expectedUser1[] = { 95, 595, 1095, -1 };
int expectedUser2[] = { 245, 745, 1245, -1 };
int expectedUser3[] = { 495, 1495, -1 };

#elif Bench_Choice == 2

#define T1_EXEC                         pdMS_TO_TICKS(95)
#define T1_PERIOD                   pdMS_TO_TICKS(250)

#define T2_EXEC                         pdMS_TO_TICKS(150)
#define T2_PERIOD                   pdMS_TO_TICKS(500)

#define T3_EXEC                         pdMS_TO_TICKS(250)
#define T3_PERIOD                   pdMS_TO_TICKS(750)

int expectedUser1[] = { 95, 345, 685, 930, 1095, -1 };
int expectedUser2[] = { 245, 835, 1245, -1 };
int expectedUser3[] = { 590, 1425, -1 };

#else

#define T1_EXEC                         pdMS_TO_TICKS(100)
#define T1_PERIOD                   pdMS_TO_TICKS(500)
```

```
#define T2_EXEC                         pdMS_TO_TICKS(200)
#define T2_PERIOD                   pdMS_TO_TICKS(500)

#define T3_EXEC                         pdMS_TO_TICKS(200)
#define T3_PERIOD                   pdMS_TO_TICKS(500)

int expectedUser1[] = { 100, 600, 1100, -1 };
int expectedUser2[] = { 300, 800, 1300, -1 };
int expectedUser3[] = { 500, 1000, 1500, -1 };

#endif
/*-----------------------------------------------------------*/
/*ID's*/
uint32_t taskID1 = 10000;
uint32_t taskID2 = 20000;
uint32_t taskID3 = 30000;

int eventid = 0;

/*-----------------------------------------------------------*/

void main_DDS(void)
{
   /* Initialize DD Scheduler and Monitor Task */
   DDSInit();

   /* Create the tasks from Test Bench (Choose from 1, 2 or 3) */
   TestInit();

   /* Start the tasks and timer running. */
   vTaskStartScheduler();

   for (;;);
}

/*-----------------------------------------------------------*/

void DDSInit()
{
   // Initialize DD Scheduler Message and Reply Queue
   xMessageQueue = xQueueCreate(100, sizeof(dd_message));
   xReplyQueue = xQueueCreate(100, sizeof(uint32_t));
   xListQueue = xQueueCreate(1, sizeof(dd_task_list**));

   // Create DD Scheduler with max priority
   xTaskCreate(DD_Scheduler, "DDS", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_MAX, &xDDS);
```

```
    // Monitor Task and its timer
    xTaskCreate(Monitor_Task, "MON", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_HIGHMAX, &xMonitor);
    vTaskSuspend(xMonitor);
    xTimerMonitor = xTimerCreate("TMR_MONITOR", pdMS_TO_TICKS(MONITOR),
pdTRUE, 0, vCallbackMonitor);
    xTimerStart(xTimerMonitor, 0);
}

/*-----------------------------------------------------------*/

void TestInit(void)
{
    // Create User Tasks with min priority
    xTaskCreate(vUserTask1, "T1", configMINIMAL_STACK_SIZE, NULL, PRIORITY_MIN,
&xUserTask1);
    vTaskSuspend(xUserTask1);
    xTaskCreate(vUserTask2, "T2", configMINIMAL_STACK_SIZE, NULL, PRIORITY_MIN,
&xUserTask2);
    vTaskSuspend(xUserTask2);
    xTaskCreate(vUserTask3, "T3", configMINIMAL_STACK_SIZE, NULL, PRIORITY_MIN,
&xUserTask3);
    vTaskSuspend(xUserTask3);

    // Create Generator Tasks with min priority
    xTaskCreate(vTaskGenerator1, "G1", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_HIGH, &xTaskGenerator1);
    xTaskCreate(vTaskGenerator2, "G2", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_HIGH, &xTaskGenerator2);
    xTaskCreate(vTaskGenerator3, "G3", configMINIMAL_STACK_SIZE, NULL,
PRIORITY_HIGH, &xTaskGenerator3);

    //Create Timers
    xTimerUser1 = xTimerCreate("TMR1", T1_PERIOD, pdTRUE, 0, vCallbackTask1);
    xTimerUser2 = xTimerCreate("TMR2", T2_PERIOD, pdTRUE, 0, vCallbackTask2);
    xTimerUser3 = xTimerCreate("TMR3", T3_PERIOD, pdTRUE, 0, vCallbackTask3);

    //Start Timers
    xTimerStart(xTimerUser1, 0);
    xTimerStart(xTimerUser2, 0);
    xTimerStart(xTimerUser3, 0);
}

/*-----------------------------------------------------------*/
// DD Callback Tasks
void vCallbackTask1(TimerHandle_t xTimer){
    vTaskResume(xTaskGenerator1);
}
```

```
void vCallbackTask2(TimerHandle_t xTimer){
   vTaskResume(xTaskGenerator2);
}

void vCallbackTask3(TimerHandle_t xTimer){
   vTaskResume(xTaskGenerator3);
}

void vCallbackMonitor(TimerHandle_t xTimer){
   vTaskResume(xMonitor);
}

/*-----------------------------------------------------------*/
void Monitor_Task(void* pvParameters)
{
   TickType_t currentTick;

   for (;;) {
      currentTick = xTaskGetTickCount();
      printf("\n      \t   Monitor Task: \t\t  %d\n", (int)currentTick);
      dd_task_list** activeTasks = get_active_dd_task_list();
      dd_task_list** completedTasks = get_completed_dd_task_list();
      dd_task_list** overdueTask = get_overdue_dd_task_list();
      vTaskSuspend(NULL);
   }
}

/*-----------------------------------------------------------*/

void vUserTask1(void* pvParameters)
{
   TickType_t currentTick;
   TickType_t prevTick;
   TickType_t executeTick;

   int* expected = &expectedUser1;

   for (;; ) {
      currentTick = xTaskGetTickCount();
      executeTick = T1_EXEC;

      while (0 < executeTick) {
         prevTick = currentTick;
         currentTick = xTaskGetTickCount();
         if (currentTick != prevTick) {
            executeTick--;
         }
```

```
        }

        if (*expected != -1) {
            printf("    %d\t   Task 1 complete\t\t  %d\t\t\t  %d\n", ++eventid, (int)currentTick,
*expected);
            expected++;
        } else {
            printf("    %d\t   Task 1 complete\t\t  %d\t\t\t  \n", ++eventid, (int)currentTick);
        }
        delete_dd_task(taskID1);
    }
}

/*-----------------------------------------------------------*/

void vUserTask2(void* pvParameters)
{
    TickType_t currentTick;
    TickType_t prevTick;
    TickType_t executeTick;
    int* expected = &expectedUser2;

    for (;; ) {
        currentTick = xTaskGetTickCount();
        executeTick = T2_EXEC;

        while (0 < executeTick) {
            prevTick = currentTick;
            currentTick = xTaskGetTickCount();
            if (currentTick != prevTick) {
                executeTick--;
            }
        }

        if (*expected != -1) {
            printf("    %d\t   Task 2 complete\t\t  %d\t\t\t  %d\n", ++eventid, (int)currentTick,
*expected);
            expected++;
        } else {
            printf("    %d\t   Task 2 complete\t\t  %d\t\t\t  \n", ++eventid, (int)currentTick);
        }
        delete_dd_task(taskID2);
    }
}

/*-----------------------------------------------------------*/

void vUserTask3(void* pvParameters)
```

```
{
    TickType_t currentTick;
    TickType_t prevTick;
    TickType_t executeTick;
    int* expected = &expectedUser3;

    for (;; )
    {
        currentTick = xTaskGetTickCount();
        executeTick = T3_EXEC;

        while (0 < executeTick)
        {
            prevTick = currentTick;
            currentTick = xTaskGetTickCount();
            if (currentTick != prevTick) {
                executeTick--;
            }
        }

        if (*expected != -1)
        {
            printf("    %d\t   Task 3 complete\t\t  %d\t\t\t  %d\n", ++eventid, (int)currentTick,
*expected);
            expected++;
        }
        else
            printf("    %d\t   Task 3 complete\t\t  %d\t\t\t  \n", ++eventid, (int)currentTick);

        delete_dd_task(taskID3);
    }
}

/*-----------------------------------------------------------*/

void vTaskGenerator1(void* pvParameters)
{
    TickType_t currentTick;

    for (;; )
    {
        currentTick = xTaskGetTickCount();

        printf("    %d\t   Task 1 released\t\t  %d\t\t\t  %d\n", ++eventid, (int)currentTick,
T1_PERIOD * (taskID1 - 10000));

        if ((int)currentTick >= 1500)
            printf("");
```

```
        create_dd_task(xUserTask1, PERIODIC, ++taskID1, T1_PERIOD * (taskID1 - 10000),
T1_PERIOD);
    }
}

/*-----------------------------------------------------------*/

void vTaskGenerator2(void* pvParameters)
{
    TickType_t currentTick;

    for (;; )
    {
        currentTick = xTaskGetTickCount();

        printf("    %d\t   Task 2 released\t\t  %d\t\t\t\t  %d\n", ++eventid, (int)currentTick,
T2_PERIOD * (taskID2 - 20000));

        create_dd_task(xUserTask2, PERIODIC, ++taskID2, T2_PERIOD * (taskID2 - 20000),
T2_PERIOD);
    }
}

/*-----------------------------------------------------------*/

void vTaskGenerator3(void* pvParameters)
{
    TickType_t currentTick;

    for (;; )
    {
        currentTick = xTaskGetTickCount();

        printf("    %d\t   Task 3 released\t\t  %d\t\t\t\t  %d\n", ++eventid, (int)currentTick,
T3_PERIOD * (taskID3 - 30000));

        if ((int)currentTick > 1500)
        {
            printf("");
        }
        create_dd_task(xUserTask3, PERIODIC, ++taskID3, T3_PERIOD * (taskID3 - 30000),
T3_PERIOD);
    }
}

/*-----------------------------------------------------------*/
```

```
dd_task allocateEmptyTask()
{
    dd_task emptyTask;

    emptyTask.t_handle = NULL;
    emptyTask.type = PERIODIC;
    emptyTask.task_id = 0;
    emptyTask.absolute_deadline = 0;
    emptyTask.completion_time = 0;
    emptyTask.release_time = 0;

    return emptyTask;
}

void DD_Scheduler(void* pvParameters)
{
    TickType_t currentTick = xTaskGetTickCount();

    dd_task_list* activeTaskList = malloc(sizeof(dd_task_list));
    dd_task_list* completedTaskList = malloc(sizeof(dd_task_list));
    dd_task_list* overdueTaskList = malloc(sizeof(dd_task_list));

    dd_message message_received;
    uint32_t reply;

    activeTaskList->task = allocateEmptyTask();
    completedTaskList->task = allocateEmptyTask();
    overdueTaskList->task = allocateEmptyTask();

    activeTaskList->next_task = NULL;
    completedTaskList->next_task = NULL;
    overdueTaskList->next_task = NULL;

    printf("Event # \t Event \t\t Measured Time (ms) \t Expected Time (ms)\n");

printf("================================================================
==========\n");

    for (;;)
{
    currentTick = xTaskGetTickCount();

    if (xQueueReceive(xMessageQueue, &message_received, portMAX_DELAY))
    {
        dd_task_list* cur, *prev;
        int flag = 1;

        currentTick = xTaskGetTickCount();
```

```
if (activeTaskList->task.t_handle != NULL) {
    vTaskSuspend(activeTaskList->task.t_handle);
    vTaskPrioritySet(activeTaskList->task.t_handle, PRIORITY_MIN);
}

switch (message_received.type)
{
case create:
{
    dd_task_list* node = malloc(sizeof(dd_task_list));
    node->task = message_received.task;
    node->next_task = NULL;

    // Check if schedulable
    if (node->task.absolute_deadline > currentTick + node->task.period)
    {
        cur= overdueTaskList;
        prev = overdueTaskList;

        dd_task_list* overdue_node = malloc(sizeof(dd_task_list));
        overdue_node->task = activeTaskList->task;
        overdue_node->next_task = NULL;

        while (cur->next_task != NULL)
        {
            prev = cur;
            cur= cur->next_task;
        }

        cur->next_task = overdue_node;
    }
    else
    {
        cur= activeTaskList;
        prev = activeTaskList;

        // Traverse list
        while (cur->next_task != NULL)
        {
            if (node->task.absolute_deadline < cur->task.absolute_deadline)
            {
                node->next_task = cur;
                if (prev == cur)
                {
                    activeTaskList = node;
                }
                else
```

```
            {
                prev->next_task = node;
            }
            flag = 0;
            break;
        }
        prev = cur;
        cur= cur->next_task;
    }

    if (flag)
    {
        if (cur->task.absolute_deadline == 0)
        {
            activeTaskList = node;
        }
        else if (node->task.absolute_deadline < cur->task.absolute_deadline)
        {
            if (prev == cur)
            {
                node->next_task = cur;
                activeTaskList = node;
            }
            else
            {
                node->next_task = cur;
                prev->next_task = node;
            }
        }
        else
        {
            cur->next_task = node;
        }
    }
}

xQueueSend(xReplyQueue, &reply, portMAX_DELAY);
break;
}

case delete:
{
activeTaskList->task.completion_time = currentTick;

dd_task_list* node = malloc(sizeof(dd_task_list));
node->task = activeTaskList->task;
node->next_task = NULL;
```

```
    if (activeTaskList->task.completion_time < activeTaskList->task.absolute_deadline)
    {
        cur= completedTaskList;
        prev = completedTaskList;

        while (cur->next_task != NULL)
        {
            prev = cur;
            cur= cur->next_task;
        }

        cur->next_task = node;
    }
    else
    {
        cur= overdueTaskList;
        prev = overdueTaskList;

        while (cur->next_task != NULL)
        {
            prev = cur;
            cur= cur->next_task;
        }

        cur->next_task = node;
    }

    cur= activeTaskList;
    prev = activeTaskList;

    if (cur->next_task == NULL)
    {
        activeTaskList->task = allocateEmptyTask();
    }
    else
    {
        activeTaskList->task = cur->next_task->task;
        activeTaskList->next_task = cur->next_task->next_task;
    }

    xQueueSend(xReplyQueue, &reply, portMAX_DELAY);
    break;
}

case active:
    xQueueSend(xListQueue, &activeTaskList, portMAX_DELAY);
    vTaskResume(xMonitor);
    break;
```

```
        case completed:
            xQueueSend(xListQueue, &completedTaskList, portMAX_DELAY);
            vTaskResume(xMonitor);
            break;

        case overdue:
            xQueueSend(xListQueue, &overdueTaskList, portMAX_DELAY);
            vTaskResume(xMonitor);
            break;

        default:
            break;
        }

        if (activeTaskList->task.absolute_deadline > 0)
        {
            if (activeTaskList->task.release_time == 0)
                activeTaskList->task.release_time = currentTick;

            vTaskPrioritySet(activeTaskList->task.t_handle, PRIORITY_HIGH);
            vTaskResume(activeTaskList->task.t_handle);
        }
    }
}
}

void create_dd_task(TaskHandle_t t_handle, task_type type, uint32_t task_id, uint32_t
absolute_deadline, uint32_t period)
{
    int reply;

    // create the dd_task struct
    dd_task task = {
        t_handle,
        type,
        task_id,
        0, // not yet released
        absolute_deadline,
        0, // not yet completed
        period
    };

    // create a task message to be created
    dd_message message = { create, task };

    // send task message to DD queue
    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
```

```
    // wait for reply back from DD scheduler
    xQueueReceive(xReplyQueue, &reply, portMAX_DELAY);

    vTaskSuspend(NULL);

}

/*-----------------------------------------------------------*/

void delete_dd_task(uint32_t task_id)
{
    int reply;
    dd_task task = { .task_id = task_id };
    dd_message message = { delete, task };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xReplyQueue, &reply, portMAX_DELAY);

}

/*-----------------------------------------------------------*/

dd_task_list** get_active_dd_task_list()
{
    dd_task_list* activelist;
    dd_message message = { .type = active };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xListQueue, &activelist, portMAX_DELAY);

    dd_task_list* cur= activelist;
    int count = 0;

    printf("      \t   Active: \t");
    while (cur->next_task != NULL)
    {
        count++;
        cur= cur->next_task;
    }

    if (cur->task.task_id != 0)
    {
        count++;
    }

    printf("%d\n", count);
    return activelist;
```

```
}

/*-----------------------------------------------------------*/

dd_task_list** get_completed_dd_task_list()
{
    dd_task_list* completedList;
    dd_message message = { .type = completed };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xListQueue, &completedList, portMAX_DELAY);

    int count = 0;
    int flag = 1;

    dd_task_list* cur= completedList;

    printf("      \t   Completed: \t");
    while (cur->next_task != NULL) {
        if (flag) {
            flag = 0;
        } else {
            count++;
        }
        cur= cur->next_task;
    }

    if (cur->task.task_id != 0){
        count++;
    }

    printf("%d\n", count);
    return completedList;
}

/*-----------------------------------------------------------*/

dd_task_list** get_overdue_dd_task_list(){
    dd_task_list* overdueList;
    dd_message message = { .type = overdue };

    xQueueSend(xMessageQueue, &message, portMAX_DELAY);
    xQueueReceive(xListQueue, &overdueList, portMAX_DELAY);

    int count = 0;
    int flag = 1;

    dd_task_list* cur= overdueList;
```

```
    printf("      \t   Overdue: \t");
    while (cur->next_task != NULL) {
        if (flag){
            flag = 0;
        } else {
            count++;
        }
        cur= cur->next_task;
    }

    if (cur->task.task_id != 0){
        count++;
    }

    printf("%d\n\n", count);

    return overdueList;
}

/*-----------------------------------------------------------*/

/* Called from prvKeyboardInterruptSimulatorTask(), which is defined in main.c. */
void vBlinkyKeyboardInterruptHandler(int xKeyPressed)
{
    /* Handle keyboard input. */
    switch (xKeyPressed)
    {
    case mainRESET_TIMER_KEY:

        if (xTimer != NULL)
        {
            /* Critical section around printf to prevent a deadlock
               on context switch. */
            taskENTER_CRITICAL();
            {
                printf("\r\nResetting software timer.\r\n\r\n");
            }
            taskEXIT_CRITICAL();

            /* Reset the software timer. */
            xTimerReset(xTimer, portMAX_DELAY);
        }

        break;

    default:
        break;
```

```
  }
}
```

A.3: Raw Test Bench #1 output

```
Event #      Event       Measured Time (ms)      Expected Time (ms)
=============================================================================
=
    1      Task 1 released        2           0
    2      Task 2 released        3           0
    3      Task 3 released        4           0
    4      Task 1 complete        99              95
    5      Task 2 complete        249             245
    6      Task 3 complete        499             495

           Monitor Task:          500
           Active:  0
           Completed:   3
           Overdue:     0

    7      Task 1 released        501             500
    8      Task 2 released        502             500
    9      Task 1 complete        598             595
    10     Task 2 complete        748             745
    11     Task 3 released        750             750

           Monitor Task:          1000
           Active:  1
           Completed:   5
           Overdue:     0

    12     Task 1 released        1002            1000
    13     Task 2 released        1002            1000
    14     Task 3 complete        1003            1495
    15     Task 1 complete        1099            1095
    16     Task 2 complete        1249            1245

           Monitor Task:          1500
           Active:  0
           Completed:   8
           Overdue:     0

    17     Task 3 released        1502            1500
    18     Task 1 released        1502            1500
    19     Task 2 released        1503            1500
    20     Task 1 complete        1599
    21     Task 2 complete        1749
    22     Task 3 complete        1999

           Monitor Task:          2000
           Active:  0
           Completed:   11
           Overdue:     0
```

A.4: Raw Test Bench #2 output

```
Event #      Event       Measured Time (ms)      Expected Time (ms)
================================================================================
=
    1      Task 1 released        2           0
    2      Task 2 released        3           0
    3      Task 3 released        4           0
    4      Task 1 complete        99              95
    5      Task 2 complete        249             245
    6      Task 1 released        250             250
    7      Task 1 complete        345             345

           Monitor Task:          500
           Active:  1
           Completed:   3
           Overdue:     0

    8      Task 2 released        501             500
    9      Task 1 released        502             500
    10     Task 3 complete        597             590
    11     Task 1 complete        692             685
    12     Task 3 released        750             750
    13     Task 1 released        750             750
    14     Task 2 complete        843             835
    15     Task 1 complete        938             930

           Monitor Task:          1000
           Active:  1
           Completed:   7
           Overdue:     0

    16     Task 2 released        1001            1000
    17     Task 1 released        1002            1000
    18     Task 1 complete        1098            1095
    19     Task 1 released        1250            1250
    20     Task 3 complete        1286            1425
    21     Task 2 complete        1436            1245

           Monitor Task:          1500
           Active:  1
           Completed:   10
           Overdue:     0

    22     Task 2 released        1502            1500
    23     Task 1 released        1502            1500
    24     Task 3 released        1503            1500
    25     Task 1 complete        1535
    26     Task 1 complete        1630
    27     Task 1 released        1750            1750
    28     Task 2     29       Task 1 complete        1875
```